Informe de Laboratorio: Implementación de Pipeline CI/CD con GitHub Actions, JUnit 5, JaCoCo, SonarCloud, Docker y Despliegue Continuo en Render

Responsable: José Luis Gil García
C.C: 1034988407
Correo: j.gil@udea.edu.co
Enlace al repositorio: https://github.com/JoseGG8/Arquisoft
Enlace a la API en producción (Render): https://arquisoft-uidq.onrender.com

---

1. Introducción

En el desarrollo de software moderno orientado a la nube, la integración y el despliegue continuos (CI/CD) constituyen el núcleo de la entrega ágil y confiable. El presente laboratorio toma como punto de partida una **API REST existente de transacciones bancarias** construida sobre Spring Boot 3 y Java 23, con el objetivo de dotarla de un flujo completo de automatización DevOps con GitHub Actions.

A lo largo del proyecto, la API pasa de ser una aplicación con validaciones manuales a un sistema gobernado por compuertas automáticas de calidad, donde cada cambio en el código fuente es auditado estáticamente, verificado a nivel de cobertura de pruebas, empaquetado como imagen inmutable en Docker Hub y desplegado sin intervención manual en la plataforma Render.

---

2. Objetivos

2.1. Objetivo General

Diseñar, estructurar e implementar un flujo automatizado de CI/CD para una API REST de transacciones bancarias, incorporando validación de pruebas unitarias, análisis estático

de código, medición de cobertura, contenedorización y despliegue continuo en la nube orquestado mediante dependencias secuenciales en GitHub Actions.

2.2. Objetivos Específicos

- Construir una batería exhaustiva de pruebas unitarias con JUnit 5 y Mockito para comprender y blindar la lógica de negocio bancaria.
- Configurar el flujo base de integración continua (`build.yml`) para ejecutar pruebas de forma automática ante eventos de control de versiones.
- Integrar SonarCloud en el flujo de trabajo para dotar al proyecto de una capa de observabilidad sobre calidad de código, deuda técnica y hotspots de seguridad.
- Instrumentar el ciclo de construcción de Maven con JaCoCo para auditar el porcentaje de cobertura de código y reflejarlo en los cuadros de mando de calidad.
- Automatizar la construcción y publicación de imágenes de contenedor en Docker Hub vinculadas a los artefactos generados.
- Orquestar el despliegue automático hacia Render mediante un webhook seguro, estableciendo dependencias estrictas entre etapas para garantizar que solo el código validado alcance el entorno productivo.

---

3. Herramientas y Tecnologías

- **Plataforma base:** Java 23 (Eclipse Temurin JDK/JRE), Spring Boot 3, Apache Maven.
- **Pruebas y Análisis:** JUnit 5, Mockito, MockMvc, JaCoCo, SonarCloud.
- **Empaquetado y Ejecución:** Docker (construcción multi-etapa), Docker Hub.
- **Infraestructura y Persistencia:** Render (PaaS), H2 Database (modo compatibilidad MySQL en memoria).
- **Orquestación y Automatización:** GitHub Actions, Git.

---

4. Metodología y Procedimiento

El desarrollo del laboratorio se planteó de manera incremental sobre la API bancaria preexistente. Cada etapa añadió una capa progresiva de madurez en el flujo de entrega, garantizando que cada eslabón dependiera del éxito del anterior.

4.1. Análisis del Dominio, Escritura y Ejecución de Pruebas Unitarias

- **Procedimiento:**
  Se tomó la API bancaria y se procedió a estudiar sus componentes transaccionales. Con el fin de comprender y validar las reglas de negocio (transferencias, validación de saldos, cuentas origen/destino y gestión de clientes), se diseñó e implementó una suite integral de 38 pruebas unitarias usando **JUnit 5**, **Mockito** y **MockMvc**.
- **Decisiones Técnicas:**
  Se tomó la decisión de aislar completamente la capa web y la capa de servicios mediante dobles de prueba (*mocks*). Esto evitó depender de instancias externas de bases de datos para correr los tests y permitió una ejecución rápida y determinista tanto en local como en los runners del pipeline.

4.2. Primer Paso en GitHub Actions: Ejecución Automática ante Eventos Push

- **Procedimiento:**
  Se inicializó el archivo `.github/workflows/build.yml` incorporando un primer job denominado `tests`. Se configuró para que, ante cualquier evento `push` sobre el repositorio, un runner de Ubuntu aprovisionara el entorno Java y ejecutara automáticamente el comando `mvn clean test`.
- **Decisiones Técnicas:**
  Se decidió incorporar la directiva nativa de caché de dependencias (`cache: 'maven'`) en la configuración del runner, optimizando los tiempos de descarga de librerías y acelerando la retroalimentación inmediata al desarrollador tras cada confirmación de código.

4.3. Capa de Observabilidad y Calidad de Código con SonarCloud

- **Procedimiento:**
  Se vinculó el repositorio de GitHub con **SonarCloud**. Se agregó en el flujo de trabajo el paso de análisis estático haciendo uso de `sonar-maven-plugin`, conectándolo de forma segura mediante el secreto `SONAR_TOKEN`. Esto habilitó un panel de control con métricas críticas del proyecto: *code smells*, duplicación de código, estimación de deuda técnica y *security hotspots*.
- **Decisiones Técnicas:**
  Para que este análisis aportara valor en el flujo colaborativo, se configuró el disparador del pipeline no solo para `push`, sino para eventos `pull_request` dirigidos a las ramas especificadas (`main` / `master`). De esta manera, cualquier rama de desarrollo que solicite integrarse pasa por una inspección estática visible antes de ser fusionada.

4.4. Incorporación de JaCoCo para Medición Objetiva de Cobertura

- **Procedimiento:**
  Para complementar la observabilidad de SonarCloud con métricas objetivas de ejecución de código, se integró la dependencia y plugin `jacoco-maven-plugin` en el archivo `pom.xml`. Se configuraron las fases `prepare-agent` y `report` dentro del ciclo de vida de Maven.
- **Decisiones Técnicas:**
  Se tomó la decisión de enlazar explícitamente el reporte generado (`jacoco.xml`) con el análisis de SonarCloud a través del parámetro `-Dsonar.coverage.jacoco.xmlReportPaths`. Esto permitió consolidar en un solo dashboard tanto el análisis estático como la cobertura de líneas e instrucciones, que alcanzó el 100% en los paquetes de negocio evaluados.

4.5. Empaquetado en Contenedores y Publicación Continua en Docker Hub

- **Procedimiento:**
  Se creó un repositorio público en **Docker Hub** para almacenar las imágenes de la aplicación. En el archivo `build.yml` se configuró el job `build` (con dependencia `needs: [tests, sonar]`), el cual compila el artefacto `.jar`, construye la imagen Docker y la publica automáticamente bajo la etiqueta `latest`.
- **Decisiones Técnicas:**
  - Las credenciales de acceso (`DOCKER_USERNAME` y `DOCKER_PASSWORD`) se resguardaron en los *Secrets* de GitHub.
  - Se optó por una estrategia de construcción multi-etapa (*multi-stage build*) en el `Dockerfile`, separando la imagen pesada de compilación de la imagen ligera de ejecución final (JRE), minimizando el peso del contenedor y la superficie de exposición.

4.6. Despliegue Continuo Desatendido en Render mediante Webhooks

- **Procedimiento:**
  Como última fase del ciclo, se configuró un servicio web en **Render** apuntando a la imagen de Docker Hub y se obtuvo su *Deploy Hook*. En el archivo `build.yml` se agregó el job final de despliegue (`deploy`), encargado de realizar una petición HTTP POST segura hacia la URL del webhook.
- **Decisiones Técnicas:**

- **Encadenamiento estricto:** El job de despliegue se configuró con `needs: build`, asegurando que el webhook solo se dispare si las pruebas unitarias, el análisis de SonarCloud y la generación de la imagen resultaron completamente exitosos.
- **Adaptación para la nube:** Para que el contenedor desplegado en Render operara de manera autosuficiente, se configuró la aplicación con persistencia relacional en memoria (H2 en dialecto MySQL) y enlace dinámico al puerto `${PORT:8080}`, garantizando disponibilidad inmediata tras el despliegue sin depender de infraestructura externa acoplada.

---

5. Resultados

1. **Pipeline de Integración y Entrega Totalmente Operativo:**
   El flujo orquestado en GitHub Actions ejecuta de manera secuencial y dependiente los cuatro jobs planificados:

$$\text{tests} \longrightarrow \text{sonar} \longrightarrow \text{build (Docker Hub)} \longrightarrow \text{deploy (Render)}$$

2. **Observabilidad y Calidad Centralizadas:**
   SonarCloud procesa automáticamente las métricas de cada Pull Request y rama principal, reportando cero incidencias bloqueantes y validando el 100% de cobertura instrumental recopilada por JaCoCo sobre las clases de negocio.
3. **Distribución Automatizada de Imágenes:**
   Cada cambio aprobado genera una nueva versión empaquetada del artefacto `.jar` y actualiza la imagen en Docker Hub sin intervención manual.
4. **Disponibilidad en Entorno de Producción:**
   La API responde en la nube de Render a través de sus rutas transaccionales:
   - **Base:** `https://arquisoft-uidq.onrender.com`
   - **Clientes:** `https://arquisoft-uidq.onrender.com/api/customers`
   - **Transacciones:** `https://arquisoft-uidq.onrender.com/api/transactions/123456`
   - **Consola de Base de Datos:** `https://arquisoft-uidq.onrender.com/h2-console`

---

6. Conclusiones

- **Construcción incremental del pipeline:** Abordar la automatización etapa por etapa (primero tests locales, luego disparadores por eventos, observabilidad de

métricas, empaquetado y finalmente despliegue) permite comprender el propósito y el valor de cada herramienta dentro de la cadena DevOps.

- **La observabilidad como criterio de aceptación:** La incorporación de SonarCloud y JaCoCo transforma la revisión de código en un proceso objetivo. Contar con un panel de calidad ante cada Pull Request ofrece al equipo visibilidad temprana sobre deuda técnica, hotspots de seguridad y cobertura antes de fusionar cambios.
- **Encadenamiento y protección del despliegue:** La directiva de dependencias (`needs`) entre jobs es la salvaguarda fundamental del Despliegue Continuo (CD). Condicionar la llamada al webhook de Render a la aprobación previa de las pruebas y la construcción de la imagen garantiza que ninguna versión inestable alcance el entorno productivo.
- **Portabilidad y desacoplamiento:** El ciclo CI/CD exige que el software sea agnóstico a la infraestructura de ejecución. Diseñar la API para configurarse dinámicamente mediante variables de entorno facilita su transición desde el runner de pruebas hasta los contenedores en plataformas PaaS.